

Big Data Programming

Tuples, Sets and Dictionaries

Chapter Overview

자료구조의 차이를 비교하고 사전 프로그램에 활용한다

튜플과 집합을 리스트와 비교한다

위치별 의미를 고정할 때는 튜플을, 중복 없는 원소와 포함 관계가 필요할 때는 집합을 사용한다. 같은 괄호 모양보다 허용되는 작업을 기준으로 구분한다.

딕셔너리의 키와 값을 연결한다

키로 값을 찾아 읽고 항목을 수정한다. 전화번호부와 영한사전으로 조회·반복·기본값을 확인하며, 다른 이름과 얕은 복사도 비교한다.

단어별 등장 횟수까지 확장한다

문자열을 단어 목록으로 만들고 딕셔너리로 횟수를 누적한다. 실습에서는 결과뿐 아니라 원본이 바뀌는 시점과 없는 키의 처리 방법을 설명한다.

Quick Review

복습: 새 목록을 만든 시점과 원본 변경 시점

In [12]:
```
values = [3, 1, 3]
ordered = sorted(values)
values.append(2)
print(values)
print(ordered)
print(values[1:3])
print(len(values), sum(values))
```

```
[3, 1, 3, 2]
[1, 3, 3]
[1, 3]
4 9
```

ordered는 append 이전에 만든 새 리스트이다. 이후 values에 2를 추가해도 ordered에는 반영되지 않는다. 이 구분은 뒤의 복사와 딕셔너리 실습에도 필요하다.

Tuples

튜플은 원소 연결을 바꾸지 않는 시퀀스이다

순서와 인덱스는 리스트와 비슷하다

튜플도 여러 객체를 순서대로 담으며 인덱싱, 슬라이싱, 반복이 가능하다. 하지만 만들어진 튜플의 원소를 다른 객체로 교체할 수는 없다.

한 원소 튜플에는 쉼표가 필요하다

(1)은 정수 1을 괄호로 묶은 식이고, (1,)은 원소 한 개의 튜플이다. 튜플을 만드는 핵심은 쉼표이며 빈 튜플은 ()로 표현한다.

함수의 여러 반환값과 연결한다

앞 단원에서 return 1, 2, 3은 튜플을 만들었다. a, b, c로 나누어 받는 과정도 튜플 원소를 순서대로 대응시키는 것이다.

생성과 접근

```
empty = ()
t = (1, 2, 3)
single = (1,)
print(t[1])
print(type((1)))
print(type(single))
a, b, c = t
print(a, b, c)
```

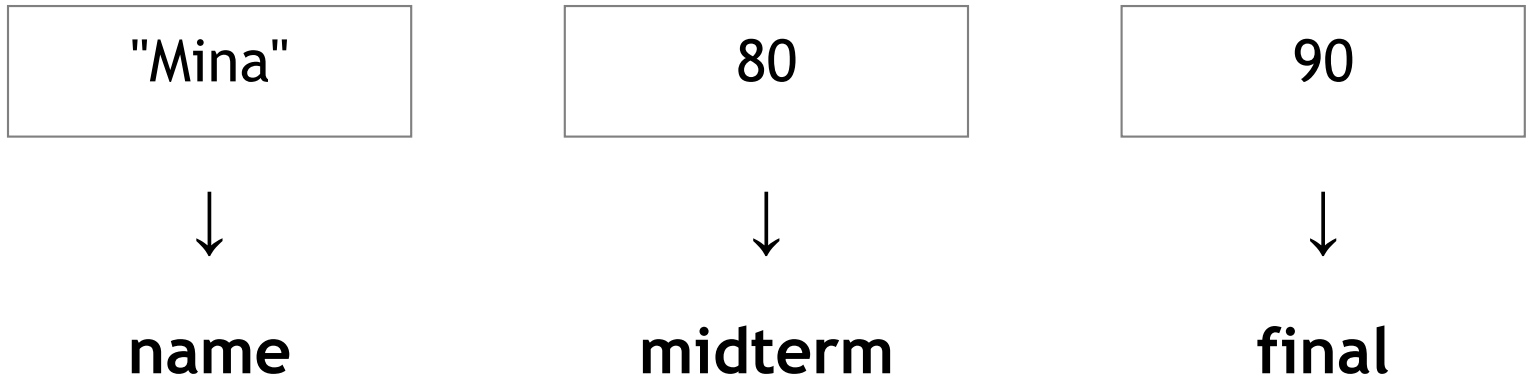
실행 결과

```
2
<class 'int'>
<class 'tuple'>
1 2 3
```

Tuples

튜플의 각 위치를 의미 있는 이름으로 나누기

record = ("Mina", 80, 90)



name, midterm, final = record

튜플의 첫 값은 name, 두 번째는 midterm, 세 번째는 final에 연결된다. 이름 세 개와 값 세 개의 순서가 일치해야 한다. 원래 튜플을 세 개로 잘라 없애는 동작은 아니다.

Tuples

불변인 것은 이름이 아니라 튜플의 원소 연결이다

튜플 항목 대입은 허용되지 않는다

t[2] = 3.14는 튜플 안의 세 번째 연결을 바꾸려는 문장이므로 TypeError가 발생한다. 같은 위치를 리스트에서 바꾸는 것은 가능하다.

이름을 새 튜플에 다시 연결할 수는 있다

t = (1, 2, 3, 4, 5)는 기존 튜플을 수정하는 것이 아니라 이름 t를 다른 객체에 연결한다. 불변 자료형도 변수 재대입을 막지는 않는다.

자료구조는 변경 규칙에 맞춰 선택한다

계속 추가·삭제할 목록에는 리스트가 자연스럽다. 위치별 의미를 고정해서 묶을 때는 튜플을 고려한다. 속도가 항상 더 빠르다는 일반화로 선택하지 않는다.

오류가 나는 튜플 대입

```
t = (23, "abc", 4.56,
     (2, 3), "def")
# 다음 문장은 TypeError
# t[2] = 3.14
```

허용되는 리스트 대입

```
li = ["abc", 23, 4.34, 23]
li[1] = 45
print(li)
```

출력: ['abc', 45, 4.34, 23]

추가로 구분할 점

튜플 내부에 리스트가 있으면 그 리스트의 내용은 바뀔 수 있다. 튜플 자신의 원소 연결과 구분한다.

Tuples

리스트 변경은 성공하고 튜플 변경은 실패한다

In [1]:
```
li = ["abc", 23, 4.34, 23]
li[1] = 45
print(li)
```

```
['abc', 45, 4.34, 23]
```

In [2]:
```
t = (23, "abc", 4.56, (2, 3), "def")
t[2] = 3.14
```

```
TypeError: 'tuple' object does not support item assignment
```

In [3]:
```
t = (1, 2, 3, 4, 5)
print(t)
```

```
(1, 2, 3, 4, 5)
```

같은 위치에 값을 대입하더라도 자료형에 따라 허용 여부가 다르다. 가운데 셀의 TypeError는 의도한 비교 예제이다. 오류 뒤의 별도 셀에서 t를 새 튜플에 연결하는 것은 가능하다.

Practice

실습 1. 성적 튜플을 해체하고 수정된 기록 만들기

위치별 의미를 이름으로 표현한다

record = ("Mina", 80, 90)의 순서는 이름, 중간 점수, 기말 점수이다. 세 변수로 나누어 받고 두 점수의 평균을 계산한다.

기존 튜플의 원소를 바꾸지 않는다

기말 점수를 95로 바꾼 새 기록 updated를 만든다. record[2]에 대입하는 것과 새로운 튜플을 만드는 것을 구분한다.

원본과 새 기록을 같이 출력한다

평균은 원래 두 점수로 계산한다. 수정된 튜플을 새 이름에 저장해도 앞에서 계산한 평균이 자동으로 바뀌지는 않는다.

완성할 코드

```
record = ("Mina", 80, 90)
name, midterm, final = _______
average = (midterm + final) / 2
updated = (name, midterm, ____)
print(name, average)
print(record)
print(updated)
```

실행 전에 확인할 내용

기존 record와 updated의 세 번째 값은 각각 무엇인가? (80)과 (80,)의 자료형도 비교한다.

Practice

해설 1. 이전 튜플과 새 튜플을 함께 유지하기

In [13]:
```
record = ("Mina", 80, 90)
name, midterm, final = record
average = (midterm + final) / 2
updated = (name, midterm, 95)
print(name, average)
print(record)
print(updated)
print(type((80)), type((80,)))
```

```
Mina 85.0
('Mina', 80, 90)
('Mina', 80, 95)
<class 'int'> <class 'tuple'>
```

평균은 85.0이다. record는 기말 점수 90을 유지하고 updated에는 95가 들어 있다. 정수 식 (80)과 한 원소 튜플 (80,)은 다르다.

Sets

집합은 중복 없는 원소의 모음이다

위치보다 포함 여부가 중요하다

집합은 원소의 중복을 허용하지 않으며 인덱스로 첫 번째나 두 번째 원소를 고르지 않는다. 입력 순서나 정렬 순서를 유지하는 자료구조가 아니다.

중괄호와 set()으로 만든다

{2, 1, 3}은 집합이다. 빈 집합은 set()으로 만들며, {}는 빈 딕셔너리이므로 구분한다. 리스트나 튜플을 set()에 전달할 수도 있다.

집합 원소는 해시 가능해야 한다

정수, 문자열과 적절한 튜플은 원소가 될 수 있다. 리스트 자체는 집합 원소로 넣을 수 없다. 지금은 포함 검사와 중복 제거의 용도에 집중한다.

생성과 중복 확인

```
numbers = {2, 1, 3, 1}
print(len(numbers))
print(2 in numbers)
print(type(set()))
print(type({}))
```

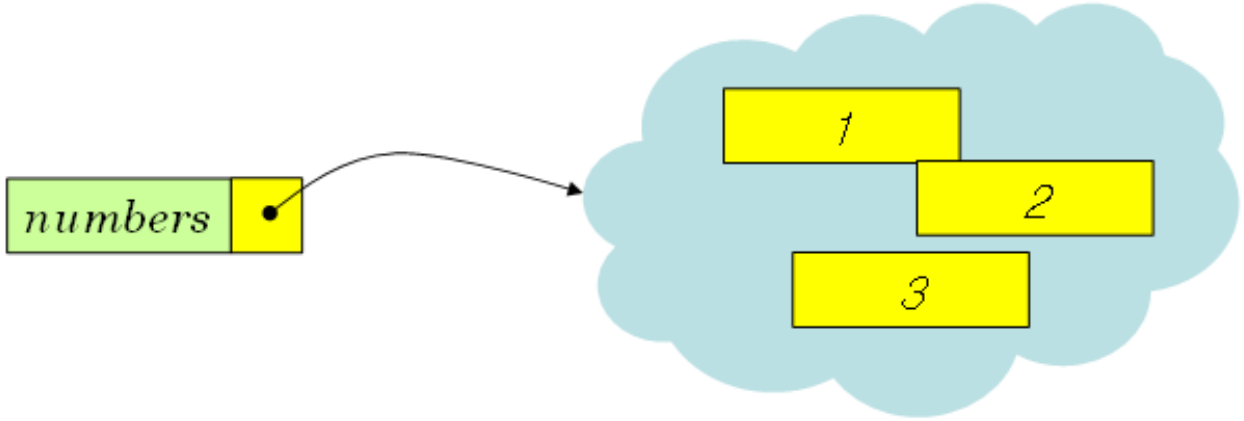
출력: 3
True
<class 'set'>
<class 'dict'>

화면 순서를 외우지 않는다

집합이 {1, 2, 3}으로 출력되어도 정렬 보장은 아니다. 필요하면 sorted()로 표시할 목록을 만든다.

Sets

그림으로 보기: 집합은 원소의 위치를 저장하지 않는다



원소가 있는지가 핵심이다

numbers에 1, 2, 3이 있다는 사실을 표현한다. 그림에서 원소의 배치 위치는 리스트의 인덱스처럼 읽지 않는다.

같은 원소를 다시 넣어도 하나이다

3이 이미 있는 집합에 3을 추가해도 길이는 늘지 않는다. 반복해서 등장한 횟수를 저장하는 목적이라면 다른 구조가 필요하다.

출력 순서를 외우지 않는다

{1, 2, 3}처럼 보여도 오름차순 정렬을 보장하지 않는다. 정렬된 표시가 필요하면 sorted()로 새 리스트를 만든다.

Sets

중복 제거와 원소 추가의 결과

목록의 길이와 종류의 수는 다르다

[1, 2, 3, 1, 2, 3]에는 원소 여섯 개가 있지만 서로 다른 값은 세 개이다. set()으로 바꾸면 같은 값이 하나씩만 남는다.

add()로 새 원소를 넣는다

prices_set.add(5555)는 없던 값을 추가한다. 이미 있는 85000을 다시 add()해도 같은 원소가 두 개로 늘어나지 않는다.

다시 리스트로 바꿔도 원래 순서는 복구되지 않는다

list(set(item_list))는 중복 없는 목록을 만들지만 최초 등장 순서를 보장하지 않는다. 순서가 필요한 문제인지 먼저 판단한다.

중복 개수 예

```
item_list = [1, 2, 3, 1, 2, 3]
item_set = set(item_list)
print(len(item_list))
print(len(item_set))
```

출력: 6
3

집합에 원소 추가

```
prices_set = {85000, 140000, 200000}
prices_set.add(5555)
prices_set.add(85000)
print(len(prices_set))
```

출력: 4

Sets

실행 화면: 목록의 원소 수와 서로 다른 값의 수

In [14]:
```
item_list = [1, 2, 3, 1, 2, 3]
item_set = set(item_list)
print(len(item_list))
print(len(item_set))
print(sorted(item_set))
item_set.add(3)
print(len(item_set))
print(item_list)
```

```
6
3
[1, 2, 3]
3
[1, 2, 3, 1, 2, 3]
```

리스트의 길이는 6이지만 집합의 길이는 3이다. 같은 원소 3을 다시 추가해도 집합의 크기는 그대로이며, 원래 item_list의 중복도 그대로 남아 있다.

Set Operations

합집합·교집합·차집합을 구분한다

같은 두 집합에 서로 다른 질문을 한다

A = {1, 2, 3}, B = {3, 4, 5}에서 둘 중 어디에라도 있는 원소, 양쪽에 공통으로 있는 원소, A에만 있는 원소를 각각 구한다.

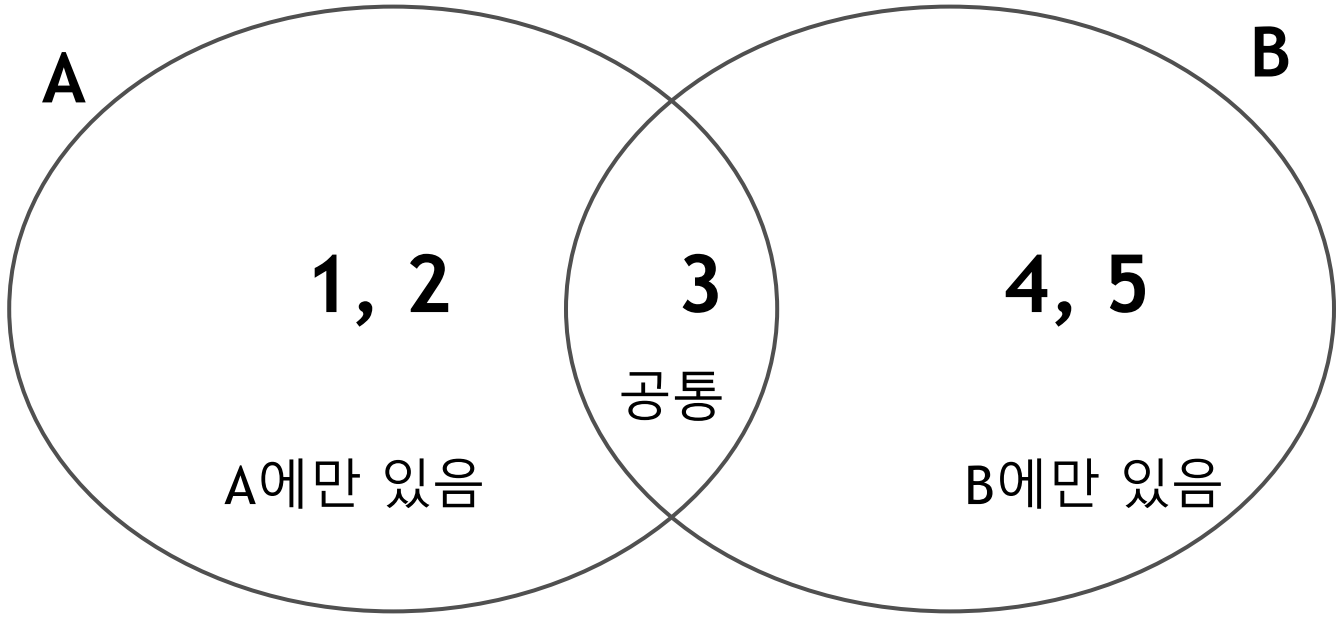
연산	묻는 내용	원소 구성
A | B	둘 중 하나 이상에 있는가?	{1, 2, 3, 4, 5}
A & B	두 집합에 모두 있는가?	{3}
A - B	A에는 있고 B에는 없는가?	{1, 2}
B - A	B에는 있고 A에는 없는가?	{4, 5}

차집합은 순서를 바꾸면 결과가 달라진다

A - B와 B - A는 다른 질문이다. 이 연산들은 새 집합을 만들며, 계산만 했다고 A나 B의 원소가 지워지지는 않는다. 표의 원소 나열 순서는 설명을 위한 것이다.

Set Operations

그림으로 읽기: 공통 원소와 양쪽의 차이



질문에 따라 선택하는 영역이 달라진다

A & B는 가운데 3만, A - B는 왼쪽 1과 2만 선택한다. 합집합 A.union(B)는 양쪽 어디에라도 있는 1, 2, 3, 4, 5를 모은다. 이 계산으로 원래 A와 B의 원소가 지워지는 것은 아니다.

Sets

중복 개수와 집합 연산의 실행 결과

In [1]:
```
item_list = [1, 2, 3, 1, 2, 3]
item_set = set(item_list)
print(len(item_list), len(item_set))
```

```
6 3
```

In [2]:
```
A = {1, 2, 3}
B = {3, 4, 5}
print(sorted(A | B))
print(sorted(A & B))
print(sorted(A - B))
print(sorted(B - A))
```

```
[1, 2, 3, 4, 5]
[3]
[1, 2]
[4, 5]
```

첫 셀은 여섯 원소 중 서로 다른 값이 세 개임을 확인한다. 뒤의 연산 결과는 보기 편하도록 sorted()로 변환한 리스트이다. 집합 자체가 정렬된다는 뜻이 아니며 A와 B는 그대로 유지된다.

Practice

실습 2. 신청 명단과 참석 명단의 차이 찾기

이름 하나를 원소 하나로 저장한다

신청자는 Mina, Jun, Sora, Jin이고 참석자는 Jun, Sora, Ara이다. 같은 이름이 중복되지 않는 예제이다.

세 질문을 집합 연산으로 바꾼다

신청도 하고 참석도 한 사람, 신청했지만 참석하지 않은 사람, 신청 명단에는 없지만 참석한 사람을 각각 구한다.

출력 순서와 집합의 성질을 구분한다

결과를 확인하기 편하도록 sorted()로 이름순 목록을 출력한다. 집합 자체가 순서를 보존한다는 뜻은 아니다.

완성할 코드

```
registered = {"Mina", "Jun",
              "Sora", "Jin"}
attended = {"Jun", "Sora", "Ara"}
print(sorted(registered __ attended))
print(sorted(registered __ attended))
print(sorted(attended __ registered))
```

실행 전에 확인할 내용

공통 항목에는 &, 차이에는 -를 사용한다. 마지막 두 식의 왼쪽 집합이 다른 이유를 설명한다.

Practice

해설 2. 차집합의 방향을 바꾸어 비교하기

In [15]:
```
registered = {"Mina", "Jun", "Sora", "Jin"}
attended = {"Jun", "Sora", "Ara"}
print(sorted(registered & attended))
print(sorted(registered - attended))
print(sorted(attended - registered))
print(sorted(registered.union(attended)))
print(len(registered), len(attended))
```

```
['Jun', 'Sora']
['Jin', 'Mina']
['Ara']
['Ara', 'Jin', 'Jun', 'Mina', 'Sora']
4 3
```

공통 참석자는 Jun과 Sora, 미참석 신청자는 Jin과 Mina, 미신청 참석자는 Ara이다. 마지막 4와 3은 원래 두 집합의 크기가 유지됨을 보여 준다.

Dictionaries

딕셔너리는 위치가 아니라 키로 값을 찾는다

키와 값이 하나의 항목을 이룬다

딕셔너리는 key: value 쌍을 저장한다. 리스트에서 0번, 1번 위치를 찾았다면 여기서는 "two"처럼 의미를 가진 키로 값을 찾는다.

키가 정수여도 위치를 뜻하지 않는다

d[1]은 두 번째 항목이 아니라 키 1에 대응하는 값이다. 정수 키 1과 문자열 키 "1"도 서로 다른 키이다.

값에는 여러 종류의 객체를 넣을 수 있다

d["blah"]는 리스트 [1, 2, 3]을 반환한다. 딕셔너리 안에 다른 자료구조를 저장하고, 그 값을 꺼내 추가 작업을 할 수 있다.

딕셔너리

```
d = {1: "hello",
     "two": 42,
     "blah": [1, 2, 3]}
print(d[1])
print(d["two"])
print(d["blah"])
```

실행 결과

```
hello
42
[1, 2, 3]
```

대괄호의 해석은 자료형에 따른다

리스트의 정수 인덱스와 딕셔너리의 키를 같은 것으로 읽지 않는다.

Dictionaries

위치로 찾는 목록과 이름으로 찾는 대응 관계

리스트			딕셔너리		
0	→	"hello"	1	→	"hello"
1	→	42	"two"	→	42
2	→	[1, 2, 3]	"blah"	→	[1, 2, 3]

대괄호의 의미는 대상 자료형에 따라 달라진다

리스트에서 [1]은 두 번째 위치이다. 오른쪽 딕셔너리에서 [1]은 정수 키 1을 찾으므로 "hello"가 나온다. 두 항목에 우연히 같은 번호가 있어도 접근 기준은 서로 다르다.

Dictionary Keys

키는 고유하며 해시 가능해야 한다

같은 키에 다시 넣으면 값이 바뀐다

var["item"] = "apple" 뒤에 같은 키로 banana를 대입하면 항목이 두 개가 되는 것이 아니다. 기존 키가 가리키는 값만 바뀐다.

키로 사용할 수 있는 자료를 확인한다

해시 가능은 키 탐색에 사용할 해시값을 안정적으로 계산할 수 있다는 뜻이다. 정수·문자열·적절한 튜플은 가능하고 리스트·딕셔너리는 불가능하다.

단순히 불변이라는 말만으로 판단하지 않는다

튜플 안에 리스트가 있으면 그 튜플도 키로 사용할 수 없다. 이 수업에서는 문자열 키를 기본으로 쓰며, 키 조건과 값의 변경 가능성을 구분한다.

같은 키에 값 다시 지정

```
var = {}
var["item"] = "apple"
var["item"] = "banana"
print(var)
print(len(var))
```

실행 결과

```
{'item': 'banana'}
1
```

허용되는 튜플 키

```
d = {("hello", "world"): 1}
print(d[("hello", "world")])
```

실행 결과

```
1
```

Dictionary Keys

실행 화면: 키의 자료형과 포함 검사

In [16]:
```
d = {1: "integer", "1": "string"}
print(d[1])
print(d["1"])
print(len(d))
print("integer" in d)
print("integer" in d.values())
```

```
integer
string
2
False
True
```

정수 키 1과 문자열 키 "1"은 서로 다른 두 항목이다. "integer" in d는 키에서 찾기 때문에 False이고, d.values()에서 찾으면 True이다.

Dictionary Order

삽입 순서와 정렬 순서는 다르다

Python 3.7 이상은 삽입 순서를 보존한다

키를 처음 추가한 순서가 반복과 표시 순서에 반영된다. 키 이름의 오름차순이나 값의 크기 순서로 자동 정렬되는 것은 아니다.

기존 키의 값 수정은 위치를 옮기지 않는다

"two"의 값을 42에서 99로 바꾸어도 그 키의 위치는 그대로이다. 새로운 키 7을 추가하면 뒤에 놓인다.

정렬이 필요하면 명시적으로 요청한다

전화번호부를 이름순으로 출력하려면 sorted(phone_book)처럼 정렬한 키 목록을 사용한다. 서로 다른 자료형의 키가 항상 비교 가능한 것은 아니다.

추가한 순서와 수정 결과

```
d = {1: "hello", "two": 42,
     "blah": [1, 2, 3]}
d["two"] = 99
d[7] = "new entry"
print(list(d))
print(d["two"])
```

실행 결과

```
[1, 'two', 'blah', 7]
99
```

삭제 후 다시 추가하면

삭제한 키를 다시 추가하는 것은 새 삽입이므로 마지막 위치로 들어간다.

Dictionary Methods

조회·수정·추가·삭제를 구분한다

대입 대상 키의 존재 여부가 동작을 가른다

d["two"] = 99는 기존 항목을 수정하고, d[7] = "new entry"는 없던 키를 추가한다. 문법은 같지만 항목 수의 변화가 다르다.

del은 키와 값의 쌍을 제거한다

del d["two"]는 그 키에 해당하는 항목 전체를 삭제한다. 값만 빈칸으로 두거나 뒤 항목의 키를 다시 번호 매기는 것이 아니다.

없는 키를 직접 조회하면 오류가 난다

d["missing"]은 키가 없을 때 KeyError를 만든다. 키가 있는지 확인하거나, 뒤에서 배울 get()으로 대체값을 지정할 수 있다.

같은 딕셔너리를 순서대로 변경

```
d = {1: "hello", "two": 42,
     "blah": [1, 2, 3]}
d["two"] = 99
d[7] = "new entry"
del d["two"]
print(list(d))
print(len(d))
print(d[7])
```

실행 결과

```
[1, 'blah', 7]
3
new entry
```

Dictionaries

같은 키의 수정과 새로운 키의 추가

In [1]:
```
d = {1: "hello", "two": 42, "blah": [1, 2, 3]}
print(d)
```

```
{1: 'hello', 'two': 42, 'blah': [1, 2, 3]}
```

In [2]:
```
d["two"] = 99
d[7] = "new entry"
print(d)
```

```
{1: 'hello', 'two': 99, 'blah': [1, 2, 3], 7: 'new entry'}
```

In [3]:
```
del d["two"]
print(d)
print(list(d), len(d))
```

```
{1: 'hello', 'blah': [1, 2, 3], 7: 'new entry'}
[1, 'blah', 7] 3
```

첫 셀은 초기 상태, 다음 셀은 수정과 추가, 마지막 셀은 삭제 이후의 상태이다. 같은 키의 값만 바꾸면 개수는 늘지 않고, 새 키를 추가하면 개수가 하나 증가한다.

Practice

실습 3. 문구류 수량을 갱신하고 항목 삭제하기

기존 키의 값을 계산한다

stock에 pen은 10개, book은 3개가 저장되어 있다. pen 두 개가 사용되었으므로 pen의 기존 값에서 2를 뺀 결과를 다시 대입한다.

추가와 삭제를 구분한다

eraser라는 새 키에 수량 5를 저장한다. 이후 book 항목을 삭제한다. 값만 바꾸는 경우와 달리 항목 수가 변한다.

항목의 존재를 따로 확인한다

작업 후 딕셔너리와 길이, book의 존재 여부를 출력한다. 키를 삭제한 뒤 남은 키들이 0번이나 1번으로 다시 번호 매겨지지는 않는다.

완성할 코드

```
stock = {"pen": 10, "book": 3}
stock["pen"] = stock["pen"] - __
stock["eraser"] = __
del stock[_______]
print(stock)
print(len(stock))
print("book" in stock)
```

실행 전에 확인할 내용

수량의 합계와 항목 개수는 다르다. len(stock)은 수량을 모두 더한 값이 아니라 키의 개수이다.

Practice

해설 3. 수정·추가·삭제 후 키의 개수 확인

In [17]:
```
stock = {"pen": 10, "book": 3}
stock["pen"] = stock["pen"] - 2
stock["eraser"] = 5
del stock["book"]
print(stock)
print(len(stock))
print("book" in stock)
```

```
{'pen': 8, 'eraser': 5}
2
False
```

pen은 8, eraser는 5이고 항목은 두 개이다. book을 삭제했으므로 "book" in stock은 False이다. 수량 8과 5를 합한 13을 len()의 결과로 읽지 않는다.

Dictionary Views

keys(), values(), items()로 읽기

각 메서드가 보여 주는 단위가 다르다

keys()는 키, values()는 값, items()는 (키, 값) 쌍을 제공한다. 딕셔너리를 for로 바로 반복하면 기본적으로 키를 하나씩 읽는다.

items()는 두 이름으로 나누어 받을 수 있다

for key, value in d.items()는 한 쌍을 키와 값으로 나눈다. 한 이름으로 받으면 그 이름에 (키, 값) 튜플이 들어간다.

반환되는 뷰와 리스트를 구분한다

keys(), values(), items()는 현재 딕셔너리를 바라보는 뷰를 반환한다. 목록이 필요하면 list()로 바꾼다. 뷰는 원본 변화가 반영될 수 있다.

작은 딕셔너리로 확인

```
d = {"foo": 1, "bar": 2}
print(list(d.keys()))
print(list(d.values()))
print(list(d.items()))
for key, value in d.items():
    print(key, value)
```

실행 결과

```
['foo', 'bar']
[1, 2]
[('foo', 1), ('bar', 2)]
foo 1
bar 2
```

Dictionary Views

실행 화면: 뷰와 별도의 키 목록은 다르다

In [18]:
```
d = {"foo": 1, "bar": 2}
keys_view = d.keys()
keys_list = list(d.keys())
d["baz"] = 3
print(list(keys_view))
print(keys_list)
print(list(d.items()))
```

```
['foo', 'bar', 'baz']
['foo', 'bar']
[('foo', 1), ('bar', 2), ('baz', 3)]
```

keys_view는 현재 딕셔너리의 변화를 보여 주므로 baz가 나타난다. 미리 list()로 만든 keys_list는 기존 두 키만 유지한다. 뷰와 그 시점에 만든 목록을 구분한다.

Dictionary Example

전화번호부를 이름순으로 출력하기

이름과 번호의 대응을 저장한다

이 예제에서는 이름을 키로, 전화번호를 문자열 값으로 저장한다. 번호는 계산할 수량이 아니므로 앞의 0과 하이픈을 보존한다.

삽입 순서와 원하는 출력 순서를 나눈다

홍길동, 강감찬, 이순신 순으로 추가했어도 이름순 출력은 다른 순서이다. sorted()로 키를 정렬한 다음 원래 딕셔너리에서 값을 찾는다.

키로 찾으면 순서가 바뀌어도 대응이 유지된다

정렬된 키마다 phone_book[key]를 조회하므로 이름과 번호가 함께 출력된다. 이름 목록과 번호 목록을 따로 정렬해서 연결하면 안 된다.

수업용 전화번호 예제

```
phone_book = {}
phone_book["홍길동"] = "010-1234-5678"
phone_book["강감찬"] = "010-1234-5679"
phone_book["이순신"] = "010-1234-5680"

for key in sorted(phone_book):
    print(key, phone_book[key])
```

이름순 출력

```
강감찬 010-1234-5679
이순신 010-1234-5680
홍길동 010-1234-5678
```

Dictionary Example

키 목록과 이름순 출력 비교하기

In [1]:
```
phone_book = {}
phone_book["홍길동"] = "010-1234-5678"
phone_book["강감찬"] = "010-1234-5679"
phone_book["이순신"] = "010-1234-5680"
print(list(phone_book.keys()))
```

```
['홍길동', '강감찬', '이순신']
```

In [2]:
```
for key in sorted(phone_book):
    print(key, phone_book[key])
```

```
강감찬 010-1234-5679
이순신 010-1234-5680
홍길동 010-1234-5678
```

위 셀의 키 목록은 추가한 순서이고, 아래 반복문은 sorted()가 만든 이름순 목록을 사용한다. 정렬된 것은 출력에 사용할 키 목록이며 phone_book 자체의 삽입 순서를 바꾼 것이 아니다.

Copying Collections

다른 이름과 복사본은 같지 않다

단순 대입은 객체를 복사하지 않는다

alias = original은 같은 리스트에 이름 하나를 더 연결한다. alias를 통해 원소를 수정하면 original에서도 같은 변경이 보인다.

list()와 copy()는 바깥 객체를 새로 만든다

l2 = list(l1)에서 l1[0]을 다시 대입해도 l2의 첫 원소 연결은 유지된다. 딕셔너리의 d.copy()도 새 딕셔너리를 만든다.

중첩 객체까지 모두 복사한 것은 아니다

이 방식은 얕은 복사이다. 안쪽에 리스트가 있다면 복사본도 그 리스트를 함께 가리킬 수 있다. 바깥 원소 대입과 안쪽 객체 변경을 구분한다.

리스트 복사

```
l1 = [1]
l2 = list(l1)
l1[0] = 22
print(l1, l2)
```

실행 결과

```
[22] [1]
```

딕셔너리 복사

```
d = {1: 10}
d2 = d.copy()
d[1] = 22
print(d, d2)
```

실행 결과

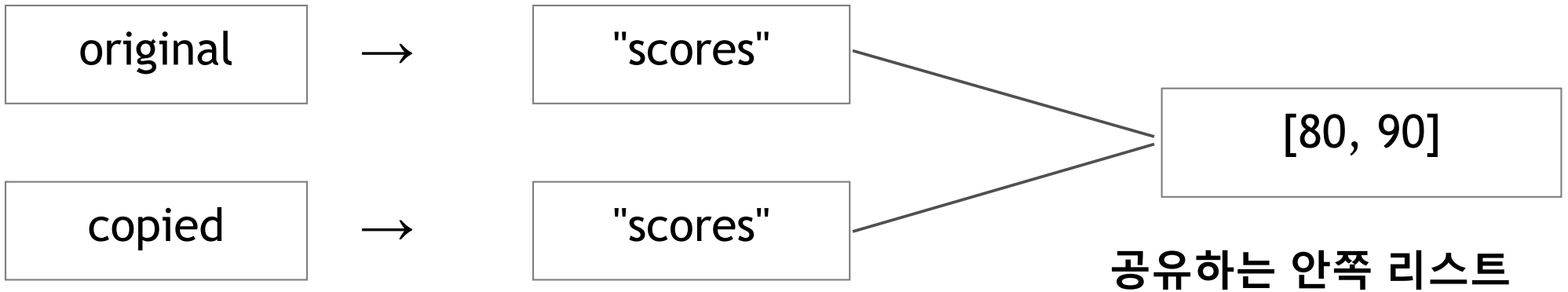
```
{1: 22} {1: 10}
```

Copying Collections

얕은 복사: 바깥은 새 객체, 안쪽은 공유할 수 있다

original = {"scores": [80, 90]}

copied = original.copy()



연결을 바꾸는가, 연결된 객체를 바꾸는가

copied["scores"].append(100)은 공유한 리스트를 변경한다. 반면 copied["scores"] = [60]은 복사본의 키가 가리키는 대상만 새 리스트로 바꾼다.

Copying Collections

바깥 복사와 안쪽 리스트 공유 확인하기

In [2]:
```
original = [[1, 2], [3, 4]]
copied = original.copy()
copied[0].append(9)
print(original)
print(copied)
```

```
[[1, 2, 9], [3, 4]]
[[1, 2, 9], [3, 4]]
```

In [3]:
```
copied[0] = [7]
print(original)
print(copied)
```

```
[[1, 2, 9], [3, 4]]
[[7], [3, 4]]
```

첫 셀은 중첩 리스트를 얕게 복사한 뒤 안쪽 리스트를 변경한 비교 예이다. copied[0].append(9)는 공유한 안쪽 리스트를 바꾼다. copied[0] = [7]은 복사본의 첫 연결만 바꾼다.

Practice

실습 4. 복사본의 변경이 원본에 보이는 경우 찾기

변경 대상이 어느 객체인지 찾는다

original을 copy()로 복사한 뒤 이름 값을 Jun으로 바꾸고 scores 리스트에 100을 추가한다. 바깥 딕셔너리의 항목 대입과 안쪽 리스트의 변경을 구분한다.

먼저 두 딕셔너리의 모습을 예상한다

copied["name"]을 바꾸어도 original의 이름은 Mina이다. scores는 어떤 결과일지 그림의 공유 관계로 설명한다.

안쪽 값을 새 목록으로 교체한다

마지막에 copied["scores"] = [60]을 실행한다. 이전 리스트의 내용을 지운 것인지, 해당 키의 연결을 바꾼 것인지 확인한다.

완성할 코드

```
original = {"name": "Mina",
            "scores": [80, 90]}
copied = original.copy()
copied["name"] = "Jun"
copied["scores"].append(100)
print(original)
print(copied)
copied["scores"] = [60]
```

실행 전에 확인할 내용

마지막 줄까지 실행한 뒤 두 scores를 각각 출력한다. 어느 변경부터 원본과 복사본의 목록이 달라지는가?

Practice

해설 4. 공유한 리스트의 변경과 연결 교체

In [19]:
```
original = {"name": "Mina", "scores": [80, 90]}
copied = original.copy()
copied["name"] = "Jun"
copied["scores"].append(100)
print(original)
print(copied)
copied["scores"] = [60]
print(original["scores"])
print(copied["scores"])
```

```
{'name': 'Mina', 'scores': [80, 90, 100]}
{'name': 'Jun', 'scores': [80, 90, 100]}
[80, 90, 100]
[60]
```

append 이후 두 딕셔너리는 같은 [80, 90, 100]을 가리킨다. [60]을 새로 대입한 뒤에는 원본 scores는 그대로이고 복사본만 [60]을 가리킨다.

Practice

실습: 영어 단어를 키로 하는 영한사전

키와 값의 역할을 먼저 정한다

영어 단어 one, two, three를 키로 사용하고, 하나, 둘, 셋을 값으로 저장한다. 사용자가 입력한 단어가 어떤 키와 같은지로 번역을 찾는다.

입력과 조회를 순서대로 연결한다

input()은 문자열을 반환하므로 단어를 숫자로 변환할 필요가 없다. 반환된 문자열을 word에 저장하고 english_dict[word]를 출력한다.

이번 실습의 입력 조건을 확인한다

먼저 등록된 세 단어만 입력한다고 가정하여 완성한다. one과 One은 다른 문자열이다. 등록되지 않은 단어는 다음 단계에서 별도로 다룬다.

완성할 코드

```
english_dict = {}
english_dict["one"] = _______
english_dict["two"] = _______
english_dict["three"] = _______
word = input("단어를 입력하시오: ")
print(_____________________)
```

출력: 단어를 입력하시오: one
하나

단어를 입력하시오: two
둘

실습 점검

키와 값을 바꾸어 저장하지 않았는지 확인한다.

Dictionary Example

그림으로 보기: 영어 단어가 조회할 키가 된다



책의 표제어처럼 키로 찾는다

영어 단어 one을 키로 정하고 한국어 뜻 하나를 값으로 저장한다. 키 자체를 번역하거나 위치 순서로 찾는 것이 아니다.

사용자 입력과 조회를 연결한다

word에 들어 있는 문자열을 english_dict[word]의 키로 사용한다. word라는 변수명과 "word"라는 문자열을 구분한다.

없는 단어는 다음 단계에서 처리한다

기본 실습은 등록된 one, two, three를 입력한다. 미등록 단어까지 허용할 때는 별도의 처리 규칙이 필요하다.

Practice

해설: 입력 문자열이 조회할 키가 된다

대괄호 안에서 word의 값을 사용한다

word가 "one"을 가리키면 english_dict[word]는 english_dict["one"]과 같은 항목을 읽는다. 문자열 "word"라는 키를 찾는 것과 다르다.

딕셔너리에 저장된 값을 결과로 사용한다

조회 결과인 "하나"가 print()에 전달된다. 키를 출력하는 것이 아니라 해당 키에 연결된 설명을 출력한다.

직접 조회의 한계를 확인한다

word가 "four"이면 등록된 키가 없어 KeyError가 발생한다. 오류가 없는 입력만 가정한 기본 예제와 미등록 단어도 처리하는 변형을 구분한다.

완성 코드

```
english_dict = dict()
english_dict["one"] = "하나"
english_dict["two"] = "둘"
english_dict["three"] = "셋"

word = input("단어를 입력하시오: ")
print(english_dict[word])
```

다른 입력으로 확인

three를 입력하면 셋을 출력한다. One을 입력하면 one과 다른 키를 조회한다.

Practice

등록된 단어와 없는 단어의 조회 결과

In [1]:
```
english_dict = {"one": "하나", "two": "둘", "three": "셋"}
```

In [2]:
```
for word in ["one", "two"]:
    print(word, english_dict[word])
```

```
one 하나
two 둘
```

In [3]:
```
word = "four"
print(english_dict[word])
```

```
KeyError: 'four'
```

위 셀은 사전을 준비하고, 다음 셀은 word에 one과 two를 차례로 넣어 조회하는 확인용 예제이다. 마지막 셀은 four를 직접 조회하여 KeyError가 발생한다. 실제 입력 실습 코드는 앞 장과 실습 Notebook에 있다.

Dictionary Methods

get()은 없는 키의 대체값을 지정한다

키가 있으면 저장된 값을 반환한다

d.get(key, default)는 키가 있으면 원래 값을 반환한다. 기본값을 썼다고 기존 값을 덮어쓰는 것이 아니다.

키가 없으면 기본값을 반환한다

기본값을 생략하면 None이 반환된다. get()을 호출한 것만으로 새 키가 등록되지는 않으므로, 없는 키가 조회 후에 추가되었다고 생각하지 않는다.

오류 처리와 자료 등록은 다른 작업이다

영한사전의 미등록 입력을 안내하려면 get(word, "등록되지 않은 단어")를 쓴다. 새 단어를 등록하려면 별도의 대입문이 필요하다.

조회 방식 비교

```
d = {"one": "하나", "two": "둘"}
print(d.get("one", "없음"))
print(d.get("four", "없음"))
print(d.get("four"))
print("four" in d)
```

실행 결과

```
하나
없음
None
False
```

영한사전 변형

```
word = "four"
print(d.get(word, "등록되지 않은 단어"))
```

Dictionary Methods

기본값 반환과 새 항목 추가는 다르다

In [1]:
```
d = {"one": "하나", "two": "둘"}
print(d.get("one", "없음"))
print(d.get("four", "없음"))
```

```
하나
없음
```

In [2]:
```
print(d.get("four"))
print("four" in d)
print(list(d.keys()))
```

```
None
False
['one', 'two']
```

one은 저장된 하나를 반환하고 four는 지정한 안내 문자열을 반환한다. 기본값을 생략한 four 조회는 None이다. 마지막 키 목록에는 여전히 one과 two만 남아 있다.

Dictionary Methods

실행 화면: 0이 저장된 키와 없는 키의 차이

In [23]:
```
stock = {"pen": 0}
print(stock.get("pen", 100))
print(stock.get("book", 100))
print(stock.get("book"))
print(stock)
```

```
0
100
None
{'pen': 0}
```

pen의 값 0은 실제로 저장된 값이므로 기본값 100으로 바뀌지 않는다. 기본값은 키가 없을 때만 사용한다. book을 조회해도 새 항목이 만들어지지 않는다.

Practice

실습 1: items()의 한 원소는 키와 값의 쌍이다

딕셔너리를 항목 목록으로 바꾼다

list(my_stocks.items())는 (키, 값) 튜플을 원소로 가진 리스트이다. 항목 네 개를 키 네 개와 값 네 개의 별도 목록으로 바꾸는 것이 아니다.

반복 변수의 이름보다 실제 값을 본다

for value in my_stocks.items()에서 value에는 숫자만 들어가지 않는다. 각 반복에서 ("ss", 10)과 같은 튜플이 들어간다.

두 이름으로 나누면 역할이 명확해진다

for name, quantity in my_stocks.items()로 받으면 키와 값을 바로 구분할 수 있다. 이 값들은 자료구조 연습용 수량이며 실제 투자 자료가 아니다.

items() 예제

```
my_stocks = {"ss": 10, "lg": 5,
             "sk": 5, "naver": 20}
my_stocks_list1 = list(my_stocks.items())
print(my_stocks_list1)
for value in my_stocks.items():
    print(value)
```

예상할 내용

목록의 첫 원소와 반복문이 네 번 출력할 값을 적는다. 키가 삽입된 순서도 함께 확인한다.

Practice

실습 2: get()의 세 가지 결과 예상하기

존재하는 키는 기본값보다 우선한다

"ss"가 있으므로 get("ss", 100)은 100이 아니라 10을 반환한다. default는 항상 반환할 값이 아니다.

없는 키에는 지정한 기본값을 사용한다

"kt"가 없으므로 get("kt", 100)은 100을 반환한다. "kt" 항목을 새로 저장하는 동작은 아니다.

기본값도 없으면 None이다

"skt"가 없고 기본값을 생략했으므로 None이 나온다. "sk"와 "skt"는 다른 키이다. 결과를 외우기보다 키 존재 여부를 먼저 판단한다.

세 조회

```
my_stocks = {"ss": 10, "lg": 5,
             "sk": 5, "naver": 20}
print(my_stocks.get("ss", 100))
print(my_stocks.get("kt", 100))
print(my_stocks.get("skt"))
print(len(my_stocks))
```

먼저 예상할 결과

세 조회 결과와 마지막 길이를 적는다. get() 호출 전후에 my_stocks가 달라졌는지 설명한다.

Practice

items()와 get()을 같은 자료에서 실행하기

In [1]:
```
my_stocks = {"ss": 10, "lg": 5, "sk": 5, "naver": 20}
for value in my_stocks.items():
    print(value)
```

```
('ss', 10)
('lg', 5)
('sk', 5)
('naver', 20)
```

In [2]:
```
print(my_stocks.get("ss", 100))
print(my_stocks.get("kt", 100))
print(my_stocks.get("skt"))
print(len(my_stocks))
```

```
10
100
None
4
```

items()는 삽입 순서대로 키와 값의 쌍을 제공한다. get()의 결과는 차례로 10, 100, None이고 항목 수는 4로 유지된다. 이름이 비슷하더라도 sk와 skt는 같은 키가 아니다.

Practice

실습 5. 여러 단어를 조회하고 미등록 단어 안내하기

기본 사전 실습을 반복으로 확장한다

영한사전에 one, two, three를 등록한다. queries에는 확인할 단어 one, four, two를 순서대로 넣는다. 여기서는 반복 조회에 집중하도록 입력 예를 리스트로 준비한다.

없는 단어는 안내 문장을 반환한다

각 word에 대해 get()으로 뜻을 찾는다. 등록된 단어의 뜻은 그대로 사용하고, 없는 단어에는 "등록되지 않은 단어"를 표시한다.

조회와 등록을 구분한다

실행 후에도 사전 항목 수는 3인지 확인한다. Four를 조회했다는 이유로 사전에 four가 추가되지는 않는다.

완성할 코드

```
english_dict = {"one": "하나",
                "two": "둘",
                "three": "셋"}
queries = ["one", "four", "two"]
for word in queries:
    meaning = english_dict.get(____,
              "등록되지 않은 단어")
    print(word, meaning)
```

실행 전에 확인할 내용

실제 키보드 입력으로 바꾸려면 word를 input()으로 받아 같은 조회문에 넣는다. 조회 원리는 바뀌지 않는다.

Practice

해설 5. get()으로 처리 범위를 넓히기

In [20]:
```
english_dict = {"one": "하나", "two": "둘", "three": "셋"}
queries = ["one", "four", "two"]
for word in queries:
    meaning = english_dict.get(word, "등록되지 않은 단어")
    print(word, meaning)
print(len(english_dict))
print("four" in english_dict)
```

```
one 하나
four 등록되지 않은 단어
two 둘
3
False
```

one과 two는 저장된 뜻을, four는 지정한 안내 문장을 출력한다. 마지막 3과 False는 사전 항목 수와 four의 미등록 상태를 확인한 결과이다.

Practice

실습 6. 단어별 등장 횟수를 딕셔너리로 세기

단어를 키, 횟수를 값으로 정한다

문장 "red blue red green blue red"를 split()으로 나눈다. 같은 단어가 다시 나오면 항목을 하나 더 만드는 대신 기존 횟수를 증가시킨다.

처음 나온 단어의 시작값을 정한다

counts에 없는 단어라면 이전 횟수를 0으로 본다. get(word, 0)으로 이전 횟수를 구하고 1을 더해 같은 키에 대입한다.

전체 개수와 종류 수를 함께 확인한다

전체 단어 수는 단어 리스트의 길이이고, 서로 다른 단어 종류의 수는 딕셔너리 키의 개수이다. 두 수가 같은지 먼저 예상한다.

완성할 코드

```
text = "red blue red green blue red"
counts = {}
for word in text.split():
    counts[word] = counts.get(____, __) + 1
print(counts)
print(len(text.split()))
print(len(counts))
```

실행 전에 확인할 내용

red가 첫 번째와 세 번째에 나왔을 때 counts["red"]가 어떻게 바뀌는지 한 단계씩 적는다.

Practice

해설 6-1. 단어를 하나 읽을 때마다 바뀌는 상태

In [22]:
```
counts = {}
for word in "red blue red green".split():
    counts[word] = counts.get(word, 0) + 1
    print(word, counts)
```

```
red {'red': 1}
blue {'red': 1, 'blue': 1}
red {'red': 2, 'blue': 1}
green {'red': 2, 'blue': 1, 'green': 1}
```

red를 처음 읽으면 0 + 1을 저장하고, 다시 읽으면 기존 횟수 1에 1을 더해 2를 저장한다. 이 화면은 상태 변화를 읽기 위해 앞의 네 단어만 실행한 비교 예제이다.

Practice

해설 6-2. 단어 수와 서로 다른 단어 수를 구분하기

In [21]:
```
text = "red blue red green blue red"
counts = {}
for word in text.split():
    counts[word] = counts.get(word, 0) + 1
print(counts)
print(len(text.split()))
print(len(counts))
print(counts.get("yellow", 0))
```

```
{'red': 3, 'blue': 2, 'green': 1}
6
3
0
```

red는 3번, blue는 2번, green은 1번 등장한다. 전체 단어는 6개, 종류는 3개이다. yellow의 조회 결과 0은 아직 등장하지 않았다는 기본값이며 새 키는 만들지 않는다.

Chapter Review

문제에 맞는 자료구조를 선택한다

자료구조	접근 기준	원소 변경·추가	대표 목적
list	정수 인덱스	가능	순서 있는 목록
tuple	정수 인덱스	원소 연결 변경 불가	고정된 위치의 묶음
set	포함 여부	가능	중복 제거와 집합 연산
dict	키	가능	이름과 값의 대응

같은 괄호보다 원하는 동작을 기준으로 선택한다

중복이 필요한지, 순서와 위치가 중요한지, 키로 값을 찾아야 하는지 먼저 판단한다. 딕셔너리는 삽입 순서를 보존하지만 위치 인덱스로 조회하는 리스트와는 다르다.

다른 구조로 변환하면 일부 정보가 달라진다

리스트를 집합으로 바꾸면 중복과 순서 정보를 그대로 유지하지 않는다. sorted()의 결과는 리스트이고 items()를 list()로 바꾸면 튜플들의 리스트를 얻는다.

Practice

마지막 확인: 결과와 원본 변화를 함께 예상하기

변경 연산과 새 결과를 구분한다

append()는 원본 리스트를 바꾼다. sorted()는 새 리스트를 만들고 set()은 중복 없는 원소의 모음을 만든다. 어느 이름에 결과를 대입했는지 살펴본다.

딕셔너리의 get()은 읽기 동작이다

없는 키의 기본값을 반환해도 키가 추가되는 것은 아니다. 조회 결과와 항목 수를 따로 예상한다.

한 줄씩 결과를 적고 확인한다

출력 여섯 줄을 예상한다. 값의 모양뿐 아니라 리스트인지 정수인지, 원본을 바꾼 문장은 무엇인지 설명한다.

종합 확인 코드

```
values = [3, 1, 3]
ordered = sorted(values)
values.append(2)
unique = set(values)
d = {"count": len(values)}
print(values)
print(ordered)
print(len(unique))
print(d.get("missing", 0))
print("missing" in d)
print(d["count"])
```

Chapter Overview

값·자료형·상태 변화를 연결해서 설명한다

종합 확인의 출력

[3, 1, 3, 2], [1, 3, 3], 3, 0, False, 4가 순서대로 출력된다. ordered는 append() 이전에 새로 만든 리스트이므로 이후 추가된 2가 자동으로 반영되지 않는다.

리스트와 튜플에서 기억할 내용

인덱스는 위치 하나를, 슬라이싱은 범위를 선택한다. 리스트는 원소를 바꿀 수 있지만 튜플은 원소 연결을 바꿀 수 없다. 변경 메서드의 반환값과 변경된 원본을 구분한다.

집합과 딕셔너리에서 기억할 내용

집합은 중복 없는 원소와 포함 관계를 다루고, 딕셔너리는 고유한 키와 값의 대응을 다룬다. 복사는 바깥 객체와 안쪽 객체의 공유 여부까지 나누어 생각한다.